Scrapy with Python -- Full Notes

1. What Is Scrapy?

Scrapy is an open-source Python framework used for web scraping, web crawling, and extracting structured data from websites.

It allows developers to collect data such as:

- Product names and prices.
- News headlines and articles.
- Blog posts.
- Job listings.
- Book information.
- Website links.
- Product reviews, where permitted.
- Publicly available structured information.

Scrapy is designed for efficient, asynchronous crawling and can handle large scraping projects.

Example

```javascript
import scrapyclass ExampleSpider(scrapy.Spider):    name = "example"    start_urls = [

"https://example.com"    ]    def parse(self, response):        yield {
"title"
: response.css("h1::text").get()        }
```

This spider requests a page and extracts its heading.

2. Why Use Scrapy?

Scrapy is useful when you need to collect data from many web pages efficiently.

Main advantages:

1. Fast and asynchronous request handling.
2. Automatic request scheduling.
3. Built-in HTML and XML selectors.
4. Pagination support.
5. Data export to JSON, JSON Lines, CSV, and other formats.
6. Item pipelines for cleaning and processing data.

7. Middleware support.
8. Retry and error-handling mechanisms.
9. Duplicate request filtering.
10. Support for large crawling projects.

3. Scrapy vs Beautiful Soup vs Selenium vs Playwright

Feature	Scrapy	Beautiful Soup	Selenium	Playwright
Main purpose	Crawling and scraping	HTML parsing	Browser automation	Browser automation
Sends HTTP requests	Yes	No	Through browser	Through browser
Parses HTML	Yes	Yes	Can access DOM	Can access DOM
JavaScript rendering	Not by default	No	Yes	Yes
Large-scale crawling	Excellent	Requires additional code	Usually less efficient	Usually less efficient
Pagination and crawling	Built in	Manual	Manual	Manual
Data pipelines	Built in	Manual	Manual	Manual
Best use	Large scraping projects	Parsing HTML	Browser testing	Modern browser testing

Important: Scrapy does not execute JavaScript in pages by default. If a website renders its content dynamically, you may need an API endpoint, a permitted HTTP-based approach, or browser integration.

4. Install Scrapy

Install using pip:

```javascript.snippet JAVASCRIPT
pip install scrapy
```

Check the version:

```javascript.snippet JAVASCRIPT
scrapy version
```

Or:

```javascript.snippet JAVASCRIPT
python -c "import scrapy; print(scrapy.__version__)"
```

It is recommended to use a virtual environment.

Create a Virtual Environment

Windows:

```javascript.snippet JAVASCRIPT
python -m venv venvvenv\Scripts\activate
```

Linux or macOS:

```javascript.snippet JAVASCRIPT
python -m venv venvsource venv/bin/activate
```

Install Scrapy:

```javascript.snippet JAVASCRIPT
pip install scrapy
```

5. Scrapy Architecture

Scrapy has several important components.

5.1 Engine

Coordinates the components of the Scrapy framework.

5.2 Scheduler

Stores requests and determines which request should be processed next.

5.3 Downloader

Sends HTTP requests and receives responses.

5.4 Spiders

Define how websites should be crawled and how data should be extracted.

5.5 Item Pipelines

Process extracted data, such as validating, cleaning, or storing it.

5.6 Downloader Middleware

Processes requests and responses around the downloader.

5.7 Spider Middleware

Processes spider inputs and outputs.

Architecture Flow

```javascript.snippet JAVASCRIPT
Spider   |   vEngine   |   vScheduler   |   vEngine   |   vDownloader   |   vResponse

|   vSpider   |   +---- Extracted Items ----> Item Pipeline   |   +---- New Requests
------->
 Scheduler
```

6. Create a Scrapy Project

Run:

```javascript.snippet JAVASCRIPT
scrapy startproject myproject
```

Move into the project directory:

```javascript.snippet JAVASCRIPT
cd myproject
```

The project structure looks like this:

```javascript.snippet JAVASCRIPT
myproject/ scrapy.cfg myproject/     __init__.py     items.py     middlewares.py
pipelines
.py     settings.py     spiders/         __init__.py         example.py
```

Important Files

- `scrapy.cfg`: Project configuration entry point.
- `items.py`: Defines item structures if needed.
- `middlewares.py`: Custom middleware.
- `pipelines.py`: Processes extracted items.
- `settings.py`: Project settings.
- `spiders/`: Contains spider classes.

7. Create Your First Spider

Create a file inside the `spiders` directory named `example.py`.

```javascript.snippet JAVASCRIPT
import scrapyclass ExampleSpider(scrapy.Spider):    name = "example"    start_urls = [

"https://example.com"    ]    def parse(self, response):        yield {
"title"
: response.css("h1::text").get(),            "url": response.url        }
```

Run the spider:

```javascript.snippet JAVASCRIPT
scrapy crawl example
```

Export the results:

```javascript.snippet JAVASCRIPT
scrapy crawl example -O output.json
```

The `-O` option overwrites the output file if it already exists.

8. What Is a Spider?

A **Spider** is a Python class that defines how Scrapy visits pages and extracts information.

Every spider typically defines:

- `name`: A unique spider name.
- `start_urls`: Initial URLs to crawl.
- `start_requests()`: An optional method for creating initial requests.
- `parse()`: A callback that processes responses.

Example:

```javascript
import scrapyclass BlogSpider(scrapy.Spider):    name = "blog"    start_urls = [

"https://example.com"    ]    def parse(self, response):        print(response.url)
```

Run:

```javascript
scrapy crawl blog
```

9. The parse() Method

The `parse()` method is a callback that processes the response returned by a request.

```javascript
def parse(self, response):    title = response.css("title::text
").get()    yield {        "
title": title    }
```

The `response` object contains useful information, including:

- `response.url`
- `response.status`
- `response.text`
- `response.body`
- `response.headers`

Example:

```javascript
def parse(self, response):    print(response.url)    print(response.status)    print(
response
.text[:500])
```

Avoid printing entire pages in large scraping projects because it can generate excessive output.

10. CSS Selectors in Scrapy

Scrapy supports CSS selectors.

Suppose the HTML is:

```javascript.snippet
Python Programming    Learn Python.Courses
```

Extract Heading Text

```javascript.snippet
response.css("h1::text").get()
```

Output:

```javascript.snippet
Python Programming
```

Extract Paragraph Text

```javascript.snippet
response.css(".description::text").get()
```

Extract Link URL

```javascript.snippet
response.css("a::attr(href)").get()
```

Output:

```javascript.snippet
/courses
```

Extract All Paragraphs

```javascript.snippet
response.css("p::text").getall()
```

Extract the Page Title

```javascript.snippet JAVASCRIPT
response.css("title::text").get()
```

11. XPath Selectors

XPath is another powerful selector language supported by Scrapy.

Extract Heading

```javascript.snippet JAVASCRIPT
response.xpath("//h1/text()").get()
```

Extract All Paragraphs

```javascript.snippet JAVASCRIPT
response.xpath("//p/text()").getall()
```

Extract Link URLs

```javascript.snippet JAVASCRIPT
response.xpath("//a/@href").getall()
```

Extract Text from a Specific Element

```javascript.snippet JAVASCRIPT
response.xpath(    "//div[@class='product']//h2/text()").get()
```

CSS vs XPath

CSS:

```javascript.snippet JAVASCRIPT
response.css(".product h2::text").get()
```

XPath:

```javascript.snippet JAVASCRIPT
response.xpath(    "//div[@class='product']//h2/text()").get()
```

Use whichever selector is clearest and most maintainable for the target HTML.

12. get() vs getall()

get()

Returns the first matching result, or `None` if no result is found.

```javascript.snippet JAVASCRIPT
title = response.css("h1::text").get()
```

getall()

Returns all matching results as a list.

```javascript.snippet JAVASCRIPT
titles = response.css("h1::text").getall()
```

Example:

```javascript.snippet JAVASCRIPT
paragraphs = response.css("p::text").getall()for paragraph in paragraphs:    print(
paragraph
)
```

These methods are commonly used when extracting data from selectors.

13. Extracting Attributes

HTML:

```javascript.snippet JAVASCRIPT
About
```

Extract href

```javascript.snippet JAVASCRIPT
response.css("a::attr(href)").get()
```

Extract Image URL

```javascript.snippet JAVASCRIPT
response.css("img::attr(src)").get()
```

Extract Image Alt Text

```javascript.snippet JAVASCRIPT
response.css("img::attr(alt)").get()
```

Extract All Links

```javascript.snippet JAVASCRIPT
links = response.css("a::attr(href)").getall()
```

14. Handling Missing Data

A selector may not match an element.

Unsafe assumption:

```javascript.snippet JAVASCRIPT
title = response.css("h1::text").get()print(title.strip())
```

If `title` is `None`, this raises an error.

Safer approach:

```javascript.snippet JAVASCRIPT
title = response.css("h1::text").get()if title:    title = title.strip()
```

Or:

```javascript.snippet JAVASCRIPT
title = (    response.css("h1::text").get(default="").strip())
```

When a value is optional, choose an appropriate default rather than assuming every page has the same structure.

15. Extracting Product Data

Suppose a page contains:

```javascript.snippet JAVASCRIPT
    Python Book    $20    View Product
```

Spider:

```javascript.snippet JAVASCRIPT
import scrapyclass ProductSpider(scrapy.Spider):    name = "products"    start_urls = [

"https://example.com"    ]    def parse(self, response):        for product in response
.
css(".product"):            yield {                "name
": product.css(                    "
.name::text"                ).get(default="").strip(),                "price
": product.css(                    "
.price::text"                ).get(default="").strip(),                "url
": product.css(                    "
a::attr(href)"                ).get()            }
```

The selectors must match the actual HTML structure of the target website.

16. Following Links

Scrapy can follow links and extract information from multiple pages.

```javascript.snippet
import scrapyclass BlogSpider(scrapy.Spider):    name = "blog"    start_urls = [

"https://example.com"    ]    def parse(self, response):        for link in response.
css
("a::attr(href)").getall():            yield response.follow(                link,

callback=self.parse_page            )    def parse_page(self, response):        yield {

"url": response.url,            "title": response.css(                "title::text"

).get()        }
```

`response.follow()` resolves relative URLs and creates a new request.

For production crawling, restrict which links you follow to avoid crawling unrelated sites or creating an unexpectedly large crawl.

17. Pagination

Pagination allows Scrapy to visit multiple pages of results.

Example HTML:

```javascript.snippet
    Next
```

Spider:

```javascript.snippet
import scrapyclass ProductSpider(scrapy.Spider):    name = "products"    start_urls = [

"https://example.com/products"    ]    def parse(self, response):        for product in

response.css(".product"):            yield {                "name
": product.css(                "
.name::text
"                ).get()            }        next_page = response.css(            "
a.next::attr(href)"        ).get()        if next_page:            yield response.
follow
(                next_page,                callback=self.parse            )
```

This pattern extracts products from each page and follows the next-page link.

18. Using start_requests()

`start_urls` is convenient for basic crawling.

For custom initial requests, use `start()` in current Scrapy versions or `start_requests()` in compatible older versions.

Example using `start()`:

```javascript
import scrapyclass CustomSpider(scrapy.Spider):    name = "custom"    def start(self):

yield scrapy.Request(            "https://example.com",            callback=self.parse

)    def parse(self, response):        yield {            "title": response.css(

"title::text"            ).get()        }
```

The exact spider lifecycle APIs depend on the Scrapy version being used. Consult the official documentation for the version installed in your environment.

19. Sending Requests with Headers

You can specify request headers when necessary.

```javascript
import scrapyclass HeaderSpider(scrapy.Spider):    name = "headers"    def start(self):

yield scrapy.Request(            "https://example.com",            headers={

"Accept": "text/html"            },            callback=self.parse        )    def
parse
(self, response):        yield {            "status": response.status        }
```

Only use headers appropriate for the request and target service. Do not use header manipulation to bypass access controls.

20. Sending POST Requests

Scrapy supports POST requests.

```javascript
import scrapyclass ApiSpider(scrapy.Spider):    name = "api"    def start(self):

yield scrapy.FormRequest(            "https://example.com/search",            formdata
={
                "query": "Python"            },            callback=self.parse        )

def parse(self, response):        yield {            "body": response.text        }
```

The form fields and endpoint must match the target application's actual requirements.

For JSON APIs, `JsonRequest` may be more suitable.

21. Scrapy Items

Items define the data fields you want to collect.

In `items.py`:

```javascript
import scrapyclass ProductItem(scrapy.Item):    name = scrapy.Field()    price = scrapy
.
Field()    url = scrapy.Field()
```

Use the item in a spider:

```javascript
import scrapyfrom myproject.items import ProductItemclass ProductSpider(scrapy.Spider):

name = "products"    start_urls = ["https://example.com
"]    def parse(self, response):        item = ProductItem()        item["
name"] = response.css(            "h1::text"        ).get()        item["price
"] = response.css(            "
.price::text"        ).get()        item["url"] = response.url        yield item
```

Items are useful when you want consistent data structures across a project.

22. Item Pipelines

Pipelines process scraped items after extraction.

Common uses:

- Cleaning data.
- Validating required fields.

- Removing duplicates.
- Writing records to a database.
- Transforming prices or dates.

Example Pipeline

In `pipelines.py`:

```javascript
class CleanProductPipeline:    def process_item(self, item, spider):        name = item
.
get("name")        if name:            item["name"] = name.strip()        return item
```

Enable the pipeline in `settings.py`:

```javascript
ITEM_PIPELINES = {    "myproject.pipelines.CleanProductPipeline": 300,}
```

The number is the pipeline priority. Lower numbers run earlier.

23. Saving Data to JSON

Command:

```javascript
scrapy crawl products -O products.json
```

JSON Lines:

```javascript
scrapy crawl products -O products.jsonl
```

The `-O` option overwrites the output file.

To append to an existing supported feed file, use `-o`:

```javascript
scrapy crawl products -o products.jsonl
```

Be careful with append mode: the behavior depends on the output format, and appending to a regular JSON array is generally not equivalent to appending JSON Lines.

24. Exporting CSV

```javascript.snippet JAVASCRIPT
scrapy crawl products -O products.csv
```

Export XML:

```javascript.snippet JAVASCRIPT
scrapy crawl products -O products.xml
```

Scrapy's feed exports support several formats. Choose a format that fits the downstream application.

25. Scrapy Settings

The `settings.py` file controls project behavior.

User Agent

```javascript.snippet JAVASCRIPT
USER_AGENT = "MyScraper/1.0"
```

Identify your scraper honestly where appropriate and follow the target site's policies.

Download Delay

```javascript.snippet JAVASCRIPT
DOWNLOAD_DELAY = 2
```

Adds a delay between requests to the same download slot.

Concurrent Requests

```javascript.snippet JAVASCRIPT
CONCURRENT_REQUESTS = 8
```

Controls the maximum number of concurrent requests across the crawler.

Robots.txt

```javascript.snippet JAVASCRIPT
ROBOTSTXT_OBEY = True
```

This setting tells Scrapy to respect robots.txt rules.

Retry Requests

```javascript.snippet JAVASCRIPT
RETRY_ENABLED = True
```

Logging Level

```javascript.snippet JAVASCRIPT
LOG_LEVEL = "INFO"
```

Settings should be adjusted based on the target site, server capacity, and applicable rules.

26. Request and Response Objects

Request

A request represents an HTTP request that Scrapy should make.

```javascript.snippet JAVASCRIPT
yield scrapy.Request(    url="https://example.com",    callback=self.parse)
```

Response

A response contains the result of an HTTP request.

```javascript.snippet JAVASCRIPT
def parse(self, response):    print(response.url)    print(response.status)    print(
response
.headers)
```

Useful response properties include:

- `response.url`
- `response.status`
- `response.headers`
- `response.text`

- `response.body`
- `response.request`

27. Handling HTTP Errors

Scrapy can receive responses such as 404 or 500.

Example:

```javascript
def parse(self, response):    if response.status != 200:        self.logger.warning(

"Unexpected status: %s",            response.status        )        return    yield {

"title": response.css(            "title::text"        ).get()    }
```

For advanced workflows, use an `errback` to process request failures.

```javascript
import scrapyclass ErrorSpider(scrapy.Spider):    name = "errors"    start_urls = [

"https://example.com"    ]    def start(self):        yield scrapy.Request(
self
.start_urls[0],            callback=self.parse,            errback=self.handle_error

)    def parse(self, response):        yield {            "status": response.status

}    def handle_error(self, failure):        self.logger.error(
"Request failed: %s"
,            failure.value        )
```

HTTP status responses and network-level failures are different. Configure error handling according to the behavior you need.

28. Logging

Use Scrapy's logger:

```javascript
def parse(self, response):    self.logger.info(        "Scraping %s",        response.
url
    )
```

Other logging levels include:

```javascript.snippet JAVASCRIPT
self.logger.debug("Debug message")self.logger.warning("Warning message
")self.logger.error("
Error message")
```

Logging is useful for debugging and monitoring crawlers.

29. Duplicate Request Filtering

Scrapy's scheduler normally filters duplicate requests based on request fingerprints.

For example, following the same URL repeatedly generally does not cause that URL to be downloaded indefinitely.

Avoid disabling duplicate filtering without a clear reason. It can significantly increase traffic and create repeated work.

30. Download Middleware

Downloader middleware processes requests and responses around the downloader.

It can be used for:

- Custom request headers.
- Request or response processing.
- Handling certain download errors.
- Integrating custom download behavior.

Middleware methods include:

```javascript.snippet JAVASCRIPT
process_request()process_response()process_exception()
```

The exact behavior and method signatures depend on the middleware API and Scrapy version.

31. Spider Middleware

Spider middleware processes inputs and outputs around the spider.

Common methods include:

```javascript.snippet
process_spider_input()process_spider_output()process_spider_exception()
```

Spider middleware can transform responses before the spider receives them or process the results produced by the spider.

32. Scraping JavaScript Websites

Scrapy does not render JavaScript automatically.

Possible approaches:

1. Find a documented or publicly accessible data API that provides the required information.
2. Inspect the page's permitted network requests and identify the underlying data source.
3. Use Playwright for pages that genuinely require browser rendering.
4. Integrate browser rendering through an appropriate, maintained Scrapy extension when justified.

Use browser rendering selectively because it usually consumes more resources than ordinary HTTP requests.

33. Scrapy Shell

Scrapy Shell is useful for testing selectors interactively.

Start the shell:

```javascript.snippet
scrapy shell "https://example.com"
```

Test CSS selectors:

```javascript.snippet
response.css("h1::text").get()
```

Test XPath:

```javascript.snippet
response.xpath("//h1/text()").get()
```

Inspect links:

```javascript.snippet JAVASCRIPT
response.css("a::attr(href)").getall()
```

The shell is an efficient way to inspect the actual response before writing spider logic.

34. Common Scrapy Commands

Create a project:

```javascript.snippet JAVASCRIPT
scrapy startproject myproject
```

Create a spider:

```javascript.snippet JAVASCRIPT
scrapy genspider example example.com
```

Run a spider:

```javascript.snippet JAVASCRIPT
scrapy crawl example
```

Export JSON:

```javascript.snippet JAVASCRIPT
scrapy crawl example -O output.json
```

Open Scrapy Shell:

```javascript.snippet JAVASCRIPT
scrapy shell "https://example.com"
```

List available spiders:

```javascript.snippet JAVASCRIPT
scrapy list
```

Display settings:

```
javascript.snippet                                                    JAVASCRIPT

scrapy settings --get BOT_NAME
```

Display Scrapy version:

```
javascript.snippet                                                    JAVASCRIPT

scrapy version
```

35. Complete Example: Book Scraper

This example demonstrates the structure of a crawler using the publicly available Books to Scrape demonstration website.

Create `books.py` inside your spiders directory:

```
javascript.snippet                                                    JAVASCRIPT

import scrapyclass BooksSpider(scrapy.Spider):    name = "books"    start_urls = [

"https://books.toscrape.com/"    ]    def parse(self, response):        for book in
response
.css("article.product_pod"):            yield {                "title
": book.css(                    "
h3 a::attr(title)"                ).get(),                "price": book.css(

".price_color::text"                ).get(),                "availability": book.css(

".availability::text"                ).getall(),                "url": response.urljoin
(
                    book.css(                        "h3 a::attr(href)"

).get()                )            }        next_page = response.css(
"li.next a::attr(href)"
        ).get()        if next_page:            yield response.follow(
next_page
,                callback=self.parse            )
```

Run:

```
javascript.snippet                                                    JAVASCRIPT

scrapy crawl books -O books.json
```

This spider extracts book titles, prices, availability text, and product URLs while following pagination.

The demonstration site is intended for scraping practice. For other sites, verify permission and follow their access policies.

36. Common Scrapy Errors

ModuleNotFoundError

Install Scrapy in the active Python environment:

```javascript.snippet JAVASCRIPT
pip install scrapy
```

Spider Not Found

Check:

- The spider file is inside the project's `spiders/` directory.
- The spider class defines a `name`.
- The command is executed from the project directory.

Empty Output

Possible reasons:

- The selector is incorrect.
- The HTML structure has changed.
- The page requires JavaScript rendering.
- The response is an error page.
- The requested data is absent.

Use Scrapy Shell to inspect the response and test selectors.

403 Forbidden

The server is refusing the request. Check whether automated access is permitted, whether authentication is required, and whether you are following the site's policies. Do not attempt to bypass access controls.

Too Many Requests

Reduce concurrency and increase download delays. Follow the site's rate limits and retry guidance.

37. Best Practices

1. Test selectors in Scrapy Shell.

2. Use `get()` for a single value and `getall()` for multiple values.
3. Handle missing values safely.
4. Follow only relevant links.
5. Use appropriate download delays and concurrency.
6. Respect robots.txt and website terms.
7. Use structured items for larger projects.
8. Keep extraction logic separate from database logic.
9. Use pipelines for validation and data cleaning.
10. Log important events and failures.
11. Avoid unnecessary browser rendering.
12. Store credentials and secrets securely.
13. Monitor crawl volume and failed requests.
14. Prefer documented APIs when they are suitable and permitted.

38. Frequently Asked Interview Questions

Q1. What is Scrapy?

Scrapy is a Python framework for web scraping and crawling.

Q2. What is a Spider?

A spider defines how to start crawling, follow links, and extract data.

Q3. What is the purpose of `parse()`?

It processes a response and can yield extracted items or additional requests.

Q4. What is the difference between `get()` and `getall()`?

`get()` returns the first match or `None`. `getall()` returns all matches as a list.

Q5. What is an Item Pipeline?

A component that processes extracted items for cleaning, validation, or storage.

Q6. What is Scrapy Shell?

An interactive environment for inspecting responses and testing selectors.

Q7. What is the Scheduler?

It manages requests waiting to be downloaded.

Q8. What is Downloader Middleware?

A component that processes requests and responses around the downloader.

Q9. What is the difference between Scrapy and Beautiful Soup?

Scrapy provides a complete crawling framework, while Beautiful Soup focuses on parsing HTML and XML.

Q10. Can Scrapy scrape JavaScript websites?

Not by default. JavaScript-rendered content may require a data API or browser integration.

Q11. What is `response.follow()`?

It creates a request for a URL relative to the current response and can attach a callback.

Q12. How can Scrapy export data?

It supports feed exports such as JSON, JSON Lines, CSV, and XML.

Q13. What is `ROBOTSTXT_OBEY`?

It configures whether Scrapy follows robots.txt rules.

Q14. What is the purpose of `scrapy crawl`?

It runs a spider by name.

Q15. Why is Scrapy useful for large crawling projects?

It provides asynchronous request processing, scheduling, duplicate filtering, pipelines, and configurable concurrency.

39. Practice Exercises

Beginner:

1. Extract a page title.
2. Extract all headings.
3. Extract all links.
4. Extract image URLs and alt text.
5. Save the results as JSON.

Intermediate:

1. Extract product names and prices.
2. Follow pagination.
3. Crawl multiple detail pages.
4. Create an Item Pipeline.
5. Export data to CSV.

Advanced:

1. Store scraped data in a database.
2. Add request error handling.
3. Configure concurrency and download delays.
4. Create a custom middleware.
5. Build a structured crawler for a multi-page demonstration website.

40. Quick Revision Cheat Sheet

```javascript
import scrapyclass ExampleSpider(scrapy.Spider):    name = "example"    start_urls = [

"https://example.com"    ]    def parse(self, response):        # Extract one value

title = response.css("h1::text
").get()        # Extract multiple values        links = response.css("
a::attr(href)").getall()        yield {            "title": title,            "url":
response
.url,            "links": links        }        # Follow a link        next_page =
response
.css(            "a.next::attr(href)"        ).get()        if next_page:
yield
 response.follow(                next_page,                callback=self.parse

)
```

Run:

```javascript
scrapy crawl example -O output.json
```

Conclusion

Scrapy is a powerful Python framework for building efficient, structured web crawlers.

Learn these topics in order:

1. Installation and project setup.
2. Spiders and the `parse()` method.
3. CSS and XPath selectors.
4. Extracting text and attributes.
5. Following links and pagination.
6. Items and pipelines.

7. Settings, middleware, and error handling.
8. JSON and CSV exports.
9. Database integration.
10. Real-world crawling projects.

Official documentation: https://docs.scrapy.org/en/latest/